

Day 4 Capstone: Blue Team Field Guide

Operation Air-Gap. Defend the Tactical Payload Gateway Through the Whole Kill Chain.

Baseline. Detect. Harden. Hold.

This is the defensive playbook for the Day 4 capstone. You defend one node against the entire bootcamp's worth of attacks, all at once. The Red team will map your surface, loot your firmware, hijack your debug port, smash your stack, hijack your wire, and try to make it stick. Your job is to build the layers before the room opens fire, then detect and harden under pressure.

Read the Red Team Field Guide. Every attack there has a matching control here. Build all six layers and your node is the one still standing when the room goes quiet.

RULES OF ENGAGEMENT: ALL BETS ARE OFF (INSIDE THE WIRE)

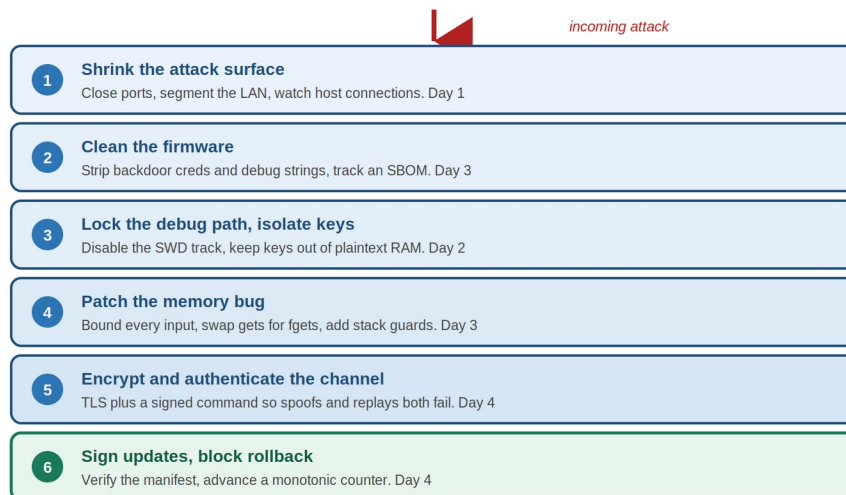
No assigned partners. No handed-out IP list. Every CЕСCP training node on the closed lab network is a valid target, and yours is one of them.

The one line you do not cross: this stays on the isolated lab subnet. Only the CЕСCP training nodes are in play. Nothing leaves this room, and no host outside the lab network is ever touched. The instructor's word ends the round, instantly.

Everything else is fair game. Hunt hard, defend harder.

Defense in Depth: One Control per Stage

The attack arrives at the top. Every layer it cannot pass is a win for your node.



Hold all six layers and your node is the one still standing when the room goes quiet.

Figure 1. Defense in depth. The attack arrives at the top. Every layer it cannot pass is a win.

Baseline first

You cannot detect a change you never measured. Before anyone attacks, record your normal

status LED pattern, your serial output, your operational loop timing, and your open ports. Every lab this week started with a baseline. The capstone is no different.

Step 0. Threat Model: STRIDE to Silicon

Before you harden, map the threats. Fill the control column for your gateway. This is the Day 1 Module 4 method, applied to your own node. It also seeds your mitigation brief at the end.

STRIDE category and where it hits your gateway	Your control (fill this in)
Spoofing: forged commands or cloned identity on the telemetry channel	_____
Tampering: firmware, boot config, or commands changed without authority	_____
Repudiation: no reliable log of who sent a command	_____
Information disclosure: keys, memory, debug output, or network traffic leaking	_____
Denial of service: a crash, flood, or jam takes the node down	_____
Elevation of privilege: debug access or a bootloader command grants control	_____

Layer 1. Shrink the Attack Surface

Counters Stage 1 recon and the Stage 5 man-in-the-middle. Source: Day 1. The less you expose, the less they can find.

1. Baseline your open ports so you know what should be listening.

```
ss -ltnp
```

What this does

`ss -ltnp` lists every TCP port your gateway is listening on, with the program behind it. Anything here that you do not need is an open door. The gateway needs only the telemetry port, and you lock that down in Layer 5.

2. Watch your own wired interface for inbound connections. You can always see traffic addressed to your own host, even on a switched network.

```
sudo tcpdump -i eth0 -A port 8080
```

What this does

`tcpdump -i eth0 -A port 8080` prints, in text, every connection that reaches your gateway port. A burst of new connections or the plaintext override is your first sign someone is hitting you.

3. Detect a man-in-the-middle. Watch your ARP table for two IP addresses that suddenly share one MAC, the tell of ARP spoofing, then pin the real pair with static entries.

```
arp -a  
sudo arp -s CONTROLLER_IP CONTROLLER_MAC
```

What this does

`arp -a` shows the current IP to MAC map. If two IPs share one MAC, someone is in the middle. `arp -s` sets a static entry that a forged ARP reply cannot overwrite, which shuts the spoof down.

A sudden scan, a new connection, or a duplicated MAC in your ARP table is the first sign someone is mapping or intercepting you. Detection buys you time to harden. On the switch, disable unused ports and segment the lab so one station cannot reach everyone.

Layer 2. Clean the Firmware

Counters Stage 2 firmware loot. Source: Day 3 Lab 3.1. Everything in a shipped binary is readable. Strip the secrets.

- Never hardcode credentials or maintenance backdoors into firmware. That is exactly what the attacker grepped out of the image.
- Strip developer and debug strings from production builds so there is nothing to mine.
- Track every component with a software bill of materials, the SBOM habit from the Day 3 debrief, so you know what is in the image you ship.

Verify like the attacker. Run the same string search against your own image. If a credential shows up for you, it shows up for them.

```
strings -n 6 your_image.bin | grep -iE "key|admin|passwd|backdoor|secret|token"
```

What this does

This is the Stage 2 attack pointed at your own build. A clean result, no credentials and no backdoor lines, means you removed the loot before the attacker could take it.

Assume the attacker already has your image. Your only safe move is to ship nothing secret inside it.

Layer 3. Lock the Debug Path and Isolate Keys

Counters Stage 3 debug hijack. Source: Day 2 debrief, Secure Enclave Engineering. An unlocked debug port is a master key.

- Disable the SWD and JTAG debug track in production, or set the option-byte lockout so any attach attempt is refused. This is the register lockout the capstone asks for.
- Isolate the key. Never leave a master key resting in plaintext global RAM. Move it into a secure element, a TPM, or a readout-protected region. This is silicon-level readout protection from the Day 2 debrief.

If you exposed a gdbserver for the network drill, close it. Rebind it to localhost so a remote attach is refused, then confirm.

```
gdbserver localhost:4444 ./auth_gate
ss -ltnp | grep 4444
```

What this does

Binding to localhost instead of 0.0.0.0 means only a debugger on your own machine can attach. This is the software version of locking the SWD track in a production device. `ss -ltnp | grep 4444` confirms the port is no longer open to the network.

Verify like the attacker. The defense passes when the debug attach is refused, or the key region no longer shows a plaintext signature. Lock the door, and do not leave the key under the mat.

Layer 4. Patch the Memory Bug

Counters Stage 4 overflow. Source: Day 3 Lab 3.2. Bound every input.

1. Open the parser source and find the unbounded read.

```
~/Desktop/CESC_Labs/Secure_Coding_Patch/main.c
```

2. Replace the unsafe call with a size-bounded one, then save.

```
// INSECURE, UNBOUNDED:
// char input_buffer[64];
// gets(input_buffer);

// SAFE, SIZE-BOUNDED:
char input_buffer[64];
fgets(input_buffer, sizeof(input_buffer), stdin);
```

What this does

`gets()` has no idea how big the buffer is, so a long payload runs off the end and corrupts the stack. `fgets()` is told the buffer size, so it stops reading at 64 bytes and the same payload is truncated safely.

3. Rebuild and flash the hardened firmware.

```
cd ~/Desktop/CESC_Labs/Secure_Coding_Patch/build
cmake -DCMAKE_BUILD_TYPE=Debug .
make
sudo openocd -f interface/cmsis-dap.cfg -f target/rp2040.cfg \
  -c "adapter speed 5000" -c "program secure_app.elf verify reset exit"
```

Add depth

One patch fixes one bug. For real hardening, add the stack protections from Module 10: stack canaries to catch an overwrite before return, MPU guard regions as no-access boundaries, separate task stacks, control-flow checks, and RTOS overflow hooks.

Verify by re-sending the 200-character payload. The defense holds when the buzzer stays silent, the prompt returns, and the input is truncated. One overflow can own a whole MCU because there is no virtual memory isolation.

Layer 5. Encrypt and Authenticate the Channel

Counters Stage 5 network injection and man-in-the-middle. Source: Day 4 Lab 4.1, Module 11. Stop trusting raw, open traffic.

1. Open the telemetry source and wrap the socket in TLS. Replace the open socket mapping in `main.c`, then save.

```
// REPLACE THE UNENCRYPTED MAPPING:
// int socket = lwip_socket(AF_INET, SOCK_STREAM, 0);

// IMPLEMENT THE HARDENED TLS SOCKET ENCAPSULATION:
tls_context_t *secure_socket = altcp_tls_new(tls_config, altcp_allocator);
```

What this does

`altcp_tls_new` wraps the raw network socket in a TLS tunnel. That adds confidentiality, so the traffic is ciphertext, and an authenticated channel, so a spoofed command from a stranger is never trusted. This is the direct fix for the MIL-STD-1553 flaw.

2. Rebuild and flash, the same pipeline as Layer 4.

```
cd ~/Desktop/CESC_Labs/Secure_Coding_Patch/build && cmake -
DCMAKE_BUILD_TYPE=Debug .. && make
sudo openocd -f interface/cmsis-dap.cfg -f target/rp2040.cfg \
-c "adapter speed 5000" -c "program secure_app.elf verify reset exit"
```

Add depth

A basic TLS tunnel stops a passive listener. To beat the man-in-the-middle and the replay from Stage 5, add the Module 11 controls: mutual TLS with ECC certificates and certificate pinning so a forged middle endpoint is rejected, the private key held in a secure element or TPM, AEAD authenticated encryption that signs each command and binds the device ID and a fresh nonce so a replayed command is refused, rate limiting, and an anomaly watch that flags the repeated override.

Verify when the attacker re-runs the injection. The defense holds when the connection is refused and your capture shows ciphertext instead of the plaintext command, even from a man-in-the-middle position.

Layer 6. Sign Updates and Block Rollback

Counters Stage 6 OTA persistence. Source: Day 4 Module 12. A signature alone is not enough.

- Only install firmware that carries a valid signed manifest. Verify the signature against a trust anchor before install and again at boot.
- Block rollback. Advance a monotonic counter, or a secure version number in eFuse, and refuse any image below the accepted version even when it is validly signed.
- Keep an A and B slot so a bad update can recover, and review the Uptane framework for the full update trust model.

Verify when the attacker tries to push a downgraded or unsigned image. The defense holds when the node refuses it. A valid signature does not prove the firmware is still safe. The counter is what stops the rollback.

Recommended Additional Defenses

Once the six layers hold, add depth from across the handbook. Each ties to a module.

- **Secure boot chain.** Anchor trust in Boot ROM, verify each stage's signature, and enforce debug restrictions and lifecycle state. (Modules 5 and 6)
- **AEAD everywhere.** Use authenticated encryption that binds metadata such as device ID and firmware version, not confidentiality alone. (Module 7)
- **Hardware-backed keys.** Store private keys in a secure element, TPM, or readout-protected flash, never in plaintext RAM. (Modules 7 and 8)
- **Memory safety in depth.** Add MPU guard regions and stack canaries so an overflow is caught before it becomes code execution. (Module 10)

- **Secure MQTT.** Require broker authentication, client identity, topic authorization, and message integrity. (Module 11)
- **Anomaly detection and rate limiting.** Flag the scan, the flood, and the repeated override, and slow the attacker down. (Modules 11 and 13)
- **Stop the man-in-the-middle.** Pin certificates so a forged middle endpoint is rejected, set static ARP entries or enable dynamic ARP inspection, and run arpwatсh to alert on a changed MAC. (Modules 11, 13, and 14)
- **Switch port security and segmentation.** Lock each switch port to a known MAC, disable unused ports, and segment the lab so one station cannot reach every gateway. (Module 13)
- **Replay and downgrade resistance.** Add a fresh nonce or counter to every command so a captured one cannot be replayed, and refuse any fallback to a weaker mode. (Module 11)
- **Signed logs for repudiation.** Keep tamper-evident records so device actions cannot be denied later. (Module 4)

Blue Team Scorecard

Mark every layer you hold. You score by surviving the kill chain, not by counting attacks. Add it up. The strongest node holds every layer and writes the brief.

Action	Points	Earned
Baseline recorded: LEDs, serial, loop, and open ports	1	
Threat model: STRIDE mapped to your gateway	2	
Layer 1: detected and blocked an ARP-spoof man-in-the-middle	2	
Layer 2: firmware cleaned, no plaintext secrets in your image	2	
Layer 3: debug path locked and key isolated, attach refused	3	
Layer 4: overflow patched, buzzer stays silent on the replay	3	
Layer 5: TLS applied, injection refused and traffic is ciphertext	3	
Layer 6: signed update and rollback block holds	3	
Survived the full melee with the node still running	4	
Mitigation brief mapped to NIST SP 800-53 and 800-160	2	
Total possible	25	

Depth wins. A node that holds every layer is the one still answering when the room goes quiet. Anyone can spray packets. Holding under fire is the engineer they are paying you to be.

Architectural Mitigation Brief

This is the capstone deliverable. For each control you applied, write one tight entry. Use only what you did in this lab, and align each fix to NIST SP 800-53 and SP 800-160. Pull the threats straight from your Step 0 table.

Example entry

Asset: the telemetry channel on the tactical gateway.

Threat (STRIDE): Spoofing and Tampering. The node trusted unauthenticated commands from any host on the subnet.

Control applied: wrapped the channel in TLS to add confidentiality and an authenticated channel.

Standard alignment: supports the transmission protection goals in NIST SP 800-53 and the systems security engineering practices in SP 800-160.

Residual risk: certificate and key lifecycle must be managed, and a flooding or replay attempt still needs an anomaly watch.

Your entries:

Instructor staging

Mirror the Red guide setup. All Pi stations on the closed wired lab subnet, the gateway listener on each Pi on port 8080, reachable Pi to Pi and blocked from outside, the Pico WH as the SWD and serial target, and no Wi-Fi. Stage the Lab 2.x, 3.1, and 3.2 targets in ~/Desktop/CESC_Labs. For the man-in-the-middle layer, install dsniff and arpwatсh on the stations. Confirm each squad records a baseline before the melee opens, and collect the mitigation brief as the graded capstone artifact.